> *[!IMPORTANT] This software is **Incubating** and subject to ECMWF's guidelines on Software Maturity.*

fdb-benchmark

The fdb-benchmark is used to simulate the quality and quantity of filesystem I/O operations in ECMWF's time-critical workflows.

The benchmark writes synthetic forecast output fields, for a number of ensemble forecast members, from a number of concurrent writer processes executing on multiple compute nodes.

At the same time, it also simulates, concurrent to the ongoing writing, the consumption of such recently written individual forecast fields by a number of reader processes fetching these fields for "product generation", "pgen" post-processing tasks; the pgen reader processes execute on different compute nodes than the writer processes.

For this, fdb-benchmark uses the same write and read methods from the fdb library, the ECMWF fields database https://github.com/ecmwf/fdb, as the production setup.

Installation

Clone the code from the GitHub repository and then checkout tag 1.0.2

```
git clone https://github.com/ecmwf/fdb-benchmark.git
cd fdb-benchmark
git checkout 1.0.2
```

Setup some build and install directories

```
# fast file system where the benchmark will be built before installing
build_root=/tmp/fdb-benchmark_$USER

# file system where FDB will store the data
fdb_root=/hpc/file/system/fdb_root

# directory (shared or not) in the compute nodes where the benchmark and
# artifacts will be installed
artifact_dir=$SCRATCH/fdb-benchmark/artifacts

# create directories

mkdir -p $artifact_dir
mkdir -p $fdb_root
mkdir -p $TMPDIR
```

Obtain the fdb source, build and install the binaries

```
./setup.sh \
    --root $build_root \
    --backend posix \
    --fdb-root $fdb_root
```

For current testing and usage of the fdb-benchmark the filesystem `MUST` be POSIX semantics (this is specificed via the `--backend posix` argument above).

If the filesystem being used is based on lustre, then the `--backend lustre` argument can be used instead. In this case the fdb-benchmark uses the lustre API to offer control over lustre stripe settings as follows:

- `.toc` and `*.index` files are automatically created with lustre stripe count 1
- `*.data` files are, by default, created with stripe count 8 and stripe size 8MB
- The above values can be adjusted by exporting environment variables: `FDB_DATA_LUSTRE_STRIPE_COUNT` and `FDB_DATA_LUSTRE_STRIPE_SIZE` around line 593 in `fdbh_one_node.sh`

Running fdb-benchmark

How fdb-benchmark works

The fdb-benchmark is orchestrated by `fdb-benchmark.sh`, which parses the user parameters to prepare the run environment, expand nodelists and distribute needed information to all nodes to run the benchmark on. It then sets up a set of arguments to be passed to `fdbh_one_node.sh`, which is finally launched on each node via SSH.

`fdbh_one_node.sh` takes the arguments and executes the benchmark workload on that node by running parallel instances of the `fdb-hammer` binary, handling CPU pinning and work distribution.

After all nodes finish the execution of `fdbh_one_node.sh`, `fdb-benchmark.sh` collects output, aggregates results and prints summary statistics.

Two separate instances of `fdb-benchmark.sh` need to be run concurrently, with one handling the setup and running of the writer processes and another handling the setup and running of the reader processes.

Setting up the sets of writer and reader nodes

The fdb-benchmark has been tested using the `nodeset` linux utility and the instructions below assume its availability. If `nodeset` isn't available or you have a different preferred utility, please consult its instructions for functionality.

```
# Get the complete list of all nodes
ALL=$JOB_NODELIST  # Adjust for the scheduler used

>NOTE: the nodelist must contain fully qualified host names

# Slice a single node off to be used for a preliminary installation run
ONE=$(nodeset --slice 1 -f $ALL)
```

The test is best run when the set of writer nodes is evenly distributed across the full set of available nodes, as this replicates the way an operational ensemble is run. This is not a requirement for testing but could be for acceptance testing.

Below are three different ways to split the set of available nodes into writer and reader node sets.

Splitting across separate groups of coupled nodes

If you have a situation where the nodes on the cluster are more connected for certain groups (e.g. nodes all on a single switch or within a leaf of a dragonfly topology), then the techniques below can be used to split the nodes into writer and reader node sets.

The following example assumes there is a large set of nodes available for benchmarking (ALL) which belong to 5 different and disjoint groups, and you want to define a set of 150 writer nodes picking the same number of nodes from each different group, and a set of reader nodes comprising the rest:

```
NUMBER_OF_GROUPS=5
NUMBER_OF_WRITER_NODES=150 # Must be a multiple of the number of groups

# Calculate the number of writers per group
WRITERS_PER_GROUP=$(( $NUMBER_OF_WRITER_NODES / $NUMBER_OF_GROUPS ))

# Expand ALL to full list of node names
ALL_NODES=$(nodeset -e "$ALL")

# Convert to array
NODE_ARR=$(echo $ALL_NODES | sed 's:,: :g')

NODE_ARRAY=($NODE_ARR)

# Initialize groups
GROUP_A=()
GROUP_B=()

# Create associative array to track counts per prefix
declare -A PREFIX_COUNTS

# Loop through all nodes
for NODE in "${NODE_ARRAY[@]}"; do
    PREFIX=${NODE:0:3}  # Adjust if needed for your naming scheme
    COUNT=${PREFIX_COUNTS[$PREFIX]:-0}
    if [[ $COUNT -lt $WRITERS_PER_GROUP ]]; then
        GROUP_A+=("$NODE")
    else
        GROUP_B+=("$NODE")
    fi
    PREFIX_COUNTS[$PREFIX]=$((COUNT + 1))
done

# Convert to nodeset format
WRITERS=$(printf "%s\n" "${GROUP_A[@]}" | nodeset -f)
READERS=$(printf "%s\n" "${GROUP_B[@]}" | nodeset -f)
```

Splitting with writer nodes spread evenly across available nodes

On large systems when the exact configuration of nodes on the network topology is unknown, then an even spread of writer nodes with nodes belonging to a member next to each other can be factored with code similar to that below:

```
# Assign sequential blocks of nodes to each member, with blocks spread evenly across
the nodelist
ALL_NODES=($(nodeset -e $ALL))
total_nodes=${#ALL_NODES[@]}
blocks=$members
block_size=$wnpm

# Get starting indices for each member, spread across the nodelist
START_INDICES=()
for i in $(seq 0 $((blocks-1))); do
    START_INDICES+=( $(( (i*total_nodes)/blocks )) )
done

WRITERS=""
for idx in "${START_INDICES[@]}"; do
    for ((j=0; j<block_size; j++)); do
        node_index=$(( (idx + j) % total_nodes ))
        WRITERS+="${ALL_NODES[$node_index]},"
    done
done
WRITERS=${WRITERS%,}  # Remove trailing comma
# Assign the rest of the nodes to readers
READERS=$(nodeset -f "$ALL" -x "$WRITERS")

# If there are more available nodes for readers than are required
# then use the pick command again to make READERS even
READERS=$(nodeset -f --pick=$wanted_readers "$READERS")
```

Splitting nodes evenly when writer nodes equals reader nodes

In the situation where the same number of writer nodes and reader nodes are used, and where these are situated is of no concern:

```
# Use nodeset split function
WRITERS=$(nodeset --split 2 -f $ALL | head -n 1)
READERS=$(nodeset --split 2 -f $ALL | tail -n 1)
```

Splitting nodes when uneven numbers of writer nodes and reader nodes

The below command picks the writer nodes randomly across all the available nodes and then splits the rest to reader nodes.

```
writernodes=$(( $writer_nodes_per_member * $num_members ))

WRITERS=$(nodeset -f --pick=$writernodes "$ALL")
READERS=$(nodeset -f "$ALL" -x "$WRITERS")

# If there are more available nodes for readers than are required then use the pick
command again to make READERS even
wanted_readers=$(( $reader_node_sets * $reader_nodes_per_step ))
READERS=$(nodeset -f --pick=$wanted_readers "$READERS")
```

Preliminary runs

Run a small single-process run to install binaries to compute nodes. It also acts as a sanity check to ensure the code is working.

```
./fdb-benchmark.sh write \
    --nodelist $ONE --ppn 1 \
    --nmembers default --nsteps 10 --nlevels 1 --nparams 1 \
    --root $build_root --config $build_root/config.yaml.in \
    --artifact-dir $artifact_dir --artifact-dir-is-shared \
    --install --verbose

# Clean the FDB afterwards
rm -rf ${fdb_root?}/rd*
```

It is recommended to do this before every main run to ensure the binaries are in the right place.

> *NOTE: if --install and --artifact-dir-is-shared, and --nodelist has more than one node, all nodes other than the first may fail due to libraries not being found*

Example SLURM Submission Scripts

The repository includes several example SLURM submission scripts that demonstrate different configurations:

- `example_submission_1member.slurm`: Basic example for testing with 1 member (9 nodes total)
- `example_submission_100members.slurm`: Large-scale example for 100 members (372 nodes total)
- `example_submission_155members.slurm`: Large-scale example for 155 members (555 nodes total)

- `consist_example_submission_1member.slurm` : Consistency checking example (9 nodes total)

These scripts show how to: - Calculate and allocate writer and reader nodes based on member count - Set up proper node assignment patterns for large member counts - Configure timing and synchronization between write and read operations - Handle consistency checking configurations

The scripts use parameterized defaults that can be overridden via command line arguments:

```
sbatch example_submission_100members.slurm [writeppn] [readppn] [members] [wnpm]
[rnpm] [readersinflight] [fdb_root]
```

Main runs

```
# --- Setup the arguments to run - must be identical for write and read

members=1
writeppn=16

rnpm=2
readppn=32

benchmark_args=" \
    --nodelist $WRITERS --ppn $writeppn \
    --nodelist-read $READERS --ppn-read $readppn \
    --nmembers $members --read-nodes-per-step $rnpm \
    --root $build_root --config $build_root/config.yaml.in \
    --artifact-dir $artifact_dir"

memberdelay=2
stepwindow=10
first_step_complete=$(($members * $memberdelay + $stepwindow + 20))
```

> *NOTE: The number of writer nodes per member is implied by the size of `$WRITERS` divided by `$members`*

```
# remove any remaining data in the fdb from previous benchmark runs
rm -rf ${fdb_root?}/rd*

# --- run contending writers and readers

./fdb-benchmark.sh write \
    $benchmark_args \
    > write.out < /dev/null &

sleep $first_step_complete

./fdb-benchmark.sh read \
    $benchmark_args \
    > read.out < /dev/null &

wait

# OPTIONAL - clean the fdb
rm -rf ${fdb_root?}/rd*
```

Changing the FDB directory for a particular run

If there is a requirement to use multiple filesystems or pools for testing, the `setup.sh` script needs to be run each time to set the desired FDB location

```
new_fdb_root=/path/to/new/fdb_root
./setup.sh --root $build_root --backend lustre --fdb_root $new_fdb_root
```

Running multiple members on one node

The number of writer nodes per member is derived from the size of the nodelist passed to `fdb-benchmark.sh` and the number of members.

- If the number of members (nmembers) is greater than the number of nodes, each node will handle multiple members. The script calculates how many members per node by dividing nmembers by the number of nodes.
- If the number of nodes is greater than the number of members, multiple nodes will handle a same member.
- The script uses these calculations to assign processes on each node to specific members, ensuring all members are covered and distributed as evenly as possible.

Command Line Arguments

Below is a summary of all input arguments for `fdb-benchmark.sh`, what they control, and their defaults.

Note: the defaults shown here are the values defined in the top of the `fdb-benchmark.sh` script; that script is the source of truth for runtime defaults. If you need to change a default permanently, update the variable in `fdb-benchmark.sh`.

`MODE`

- Options: `write`, `read`, `list`
- Specifies the operation mode: writing data, reading data, or listing data fields.
- Only `write` and `read` are needed for ITT380.

`--nodelist <list>`

- Node list (following Slurm syntax) where to run fdb-hammer processes. E.g. compute-node[001-010]. Do not use 'localhost' in this list, use the local host name if needed. **Default:** a list containing the local host name only (as provided by hostname).

`--ppn <ppn>`

- Number of fdb-hammer processes to run on every client node in the provided node list.
  Default: `1`

`--nmembers <nmembers>`

- Total number of members to archive/retrieve by all client nodes and process. It must be a multiple or submultiple of the number of nodes in the nodelist. If larger than the number of nodes, a node will produce/consume data for more than one member. If smaller, multiple nodes will produce/consume data for a same member. **Default:** one per node in --nodelist (this default behaviour can be triggered by providing no value or with --nmembers default).

`--nsteps <nsteps>`

- Number of steps to archive by every client process (if MODE is 'write') or archived by writers (if MODE is 'read'). If MODE is 'write', all processes archive fields for steps 1 to nsteps. **Default:** `90`

`--nlevels <nlevels>`

- Number of levels to archive by every client process (if MODE is 'write') or archived by writers (if MODE is 'read'). If MODE is 'write', every parallel process in a member archives nlevels unique levels. **Default:** `120`

`--nparams <nparams>`

- Number of params to archive by every client process (if MODE is 'write') or archived by writers (if MODE is 'read'). If MODE is 'write', all processes archive fields for the same nparams params. **Default:** `6`

`--field-size <size>`

- Size of the GRIB field to be used as seed for all writes. **Default:** `17.37MiB`

`--no-ccsds`

- Flag to disable CCSDS compression. **Default (script):** CCSDS compression is enabled by default (`ccsds=yes`).

`--no-randomise-data`

- Flag to disable field data randomisation (if MODE is 'write'). By default, the data of every field written is randomised with decimal values between 0 and 1. **Default (script):** Field data randomisation is enabled by default (`randomise_data=yes`).

`--itt` / `--no-itt`

- Flag to enable/disable ITT mode, where the writers barrier at the end of every step, and the readers poll the FDB until their data becomes available. Readers retrieve data in a transposed way (i.e., every reader process accesses data for a single or a few time steps). When --itt is supplied and the MODE is 'read', the --nodelist, --ppn, --nmembers, --nsteps, --nlevels and --nparams options are interpreted as a description of the span of weather fields archived in the write mode. **Default:** Enabled

`--nodelist-read <list>`

- If MODE is 'read' and --itt is supplied, a list of nodes to be employed for the 'read'

mode, where to run fdb-hammer processes, must be provided via --nodelist-read, following the Slurm node list syntax. E.g. compute-node[011-020]. Do not use 'localhost' in this list, use the local host name if needed. **Default:** not set (required in ITT read mode)

`--ppn-read <ppn>`

- If MODE is 'read' and --itt is supplied, the number of fdb-hammer processes per node to run for the 'read' mode must be provided via --ppn-read. **Default:** not set (required in ITT read mode)

`--read-nodes-per-step <nnodes>`

- If --itt is specified and MODE is 'read', --read-nodes-per-step determines the number of reader nodes to employ for reading data for every written step. It must be equal or smaller than the number of nodes in --nodelist-read. If smaller, it must be a divisor. **Default:** one node in --nodelist-read per step if --nsteps is greater than or equal to the number of nodes in the nodelist, or length(--nodelist-read) / --nsteps otherwise (this default behaviour can be triggered by providing no value or with --read-nodes-per-step default).

`--barrier-port <port>`

- If --itt is specified and MODE is 'write', the port specified in --port will be used on the first writer node to listen for peer nodes to barrier. **Default:** `7777`

`--barrier-max-wait <seconds>`

- If --itt is specified and MODE is 'write', --barrier-max-wait determines the number of seconds to wait for peer nodes during barriers before aborting. **Default:** `10`

`--poll-period <period>`

- If --itt is specified and MODE is 'read', --poll-period determines the number of seconds between list/polling retries in reader processes. **Default:** `10`

`--poll-max-attempts <attempts>`

- If --itt is specified and MODE is 'read', --poll-max-attempts determines the maximum

number of list retries before failing. **Default:** `200`

`--member-delay <seconds>`

- If --itt is specified and MODE is 'write', writer processes for a given member are launched with a delay of 'seconds' seconds after the processes for the previous member. Decimal numbers supported. **Default:** `2`

`--reader-delay <seconds>`

- If --itt is specified and MODE is 'read', reader processes for a given step are launched with a delay of 'seconds' seconds after the processes for the previous step. Decimal numbers supported. **Default:** `0`

`--step-window <seconds>`

- If --itt is specified and MODE is 'write', --step-window determines the number of seconds allowed per writer process to perform the I/O for a step. If this amount of time is not consumed during I/O, the process sleeps until it is fully consumed. If the window is exceeded, the process prints a message in stdout. **Default:** `10`

`--random-delay <percent>`

- If --itt is specified and MODE is 'write', every writer process sleeps for a random amount of time between 0 and (--step-window * percent / 100) before starting I/O. **Default:** `100`

`--read-step-window <seconds>`

- If --itt is specified and MODE is 'read', --read-step-window determines the number of seconds allowed for reader processes for a given step to perform the I/O. If this amount of time is not consumed during I/O, the processes sleep until it is fully consumed. If a process exceeds the window, it prints a message in stdout. **Default:** `10`

`--read-random-delay <percent>`

- If --itt is specified and MODE is 'read', every reader process sleeps for a random amount of time between 0 and (--read-step-window * percent / 100) before starting I/

O. **Default:** `0`

`--prelist` / `--no-prelist`

- If --itt is specified and MODE is 'read', this flag enables/disables pre-listing of the locations of all fields to be read by every reader node. The first process in every reader node performs the pre-listing, splits the obtained field locations in as many subsets as --ppn-read, and every reader process is assigned one such subset for direct bulk data retrieval without listing. If this flag is disabled every reader process lists the fields of its assigned subset. **Default:** Enabled

`--root <path>`

- Path to the root directory where the FDB and other repositories and binaries have been installed. **Default:** `$HOME/fdb-benchmark`

`--config <path>`

- Path to an FDB client configuration file. This file will be deployed on all client nodes in nodelist. It can contain wildcards such as @SCHEMA_PATH@ which will be replaced by the actual schema file path on that client node. **Default:** `<root>/config.yaml.in`

`--prolog-script <path>`

- Path to a prolog script to be sourced first thing on each node in nodelist, for example to load required modules. **Default:** none

`--md-check`

- Flag to enable metadata consistency checks. The reader fdb-hammer processes become memory-hungry if this parameter is enabled, as they need to buffer all fields read for later verification. **Default:** disabled

`--full-check`

- Flag to enable metadata and data consistency checks. The reader fdb-hammer processes become memory-hungry if this parameter is enabled, as they need to buffer all fields read for later verification. This option is more compute demanding

than --md-check. **Default:** disabled

`--install`

- Flag to enable installation of fdb-hammer and other necessary binaries on the client nodes. It must be specified on the first run on a given set of client nodes, or if the binaries on these nodes need to be updated with new ones. **Default:** disabled

`--artifact-dir <path>`

- Path where to install binaries and artifacts on the client nodes. Use '~' to refer to the home directory on the nodes, but do not set artifact-dir to only '~'. **Default:** `~/fdb-benchmark/artifacts`

`--artifact-dir-is-shared` / `--no-artifact-dir-is-shared`

- Flag to be provided if the artifact directory on the client nodes is shared via a networked file system. **Default:** yes

`--verbose`

- Print field identifiers archived or retrieved. **Default:** disabled

`-h`, `--help`

- show this menu

Current Limitations

Limitations for Combinations of Nodes per Reader (ITT Read Mode)

- The number of reader nodes per step (`--read-nodes-per-step`) must be less than or equal to the total number of nodes specified in `--nodelist-read` .

- If `--read-nodes-per-step` is less than the total number of reader nodes, it must be a divisor of the total number of reader nodes.
- If the `--prelist` option is enabled, the number of levels (`--nlevels`) must be divisible by the number of reader nodes per step.
- If the number of steps (`--nsteps`) is less than the number of reader nodes, the number of reader nodes must be divisible by the number of steps.
- These constraints ensure that the workload is evenly distributed and that each reader node gets a valid subset of data to process.

If these conditions are not met, the script will abort with an error message to prevent misconfiguration and ensure correct parallel execution.

Running Consistency Checks

It is important that, not only, that the storage subsystem is able to manage the demands of the ECMWF operational workflow but also that it does so whilst ensuring the correctness of data.

Therefore, it is also required to do a separate run with consistency checks enabled that ensures the correctness of data is maintained. Enabling these checks affects performance and so no timing data is required for these runs.

The consistency checking example can be found in `consist_example_submission_1member.slurm`, which shows the proper configuration:

```
# To enable consistency checks change the benchmark_args to the following:
benchmark_args="--nodelist $WRITERS --ppn $writeppn --field-size 17.37MiB \
    --nodelist-read $READERS --ppn-read $readppn --read-nodes-per-step $rnpm \
    --nmembers $members --nsteps 90 --nlevels 120 --nparams 6 \
    --root $build_root --config $build_root/config.yaml.in \
    --artifact-dir $artifact_dir --no-itt --full-check --no-randomise-data --no-
prelist"
```

> *NOTE: This disables the random ordering of an ITT benchmark run and makes the reading deterministic. The consistency checker also requires a longer delay before starting readers to ensure sufficient writes have completed.*

```
# Example calculation for sleep time before read runs start:
# 8 * $member_delay + step-window + safety_margin
# e.g.
sleeptime=$((8 * 2 + 10 + 200))
```

Clean-up procedure

If the benchmark run hangs or terminates abruptly follow these steps to clean up. Uses the clush commands from clustershell

```
# list active writer/reader processes on the compute nodes
clush -w $ALL "ps -aux | grep '^$USER ' | wc -l"
# if any, kill as follows
#clush -w $ALL "ps -aux | grep 'fdb-hammer ' | awk '{print \$2}' | xargs -I{} kill
{}"
#clush -w $ALL "ps -aux | grep 'bash -s ' | awk '{print \$2}' | xargs -I{} kill {}"
#clush -w $ALL "ps -aux | grep 'timeout 500' | awk '{print \$2}' | xargs -I{} kill
{}"

# list active orchestrating processes on the login/orchestrating node
ps -aux | grep fdb-benchmark.sh
# if any, kill as follows
#ps -aux | grep fdb-benchmark.sh | awk '{print $2}' | xargs -I{} kill {}

# list leftover lock files in the compute nodes
clush -w $ALL 'ls /tmp/$USER/'
# if any, remove as follows
#clush -w $ALL 'rm -rf /tmp/$USER/fdb-benchmark*'

# --- other cleanup tasks, usually required if needing to start from scratch,
#     rerun with manipulated artifacts, or recompile the benchmark.
#     Sorted from less to more destructive.

# removes the FDB data directory
rm -rf ${fdb_root?}

# removes the artifacts installed by setup.sh
rm -rf ${SCRATCH?}/fdb-benchmark

# removes the benchmark build
rm -rf ${build_root?}

# removes the benchmark repository
rm -rf ${SCRATCH?}/git/fdb-benchmark
```

License

Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

In applying this licence, ECMWF does not waive the privileges and immunities granted to it by virtue of its status as an intergovernmental organisation nor does it submit to any jurisdiction.